Source: docs\operations\maintenance-troubleshooting.md | Export: 11 November 2026

Maintenance and troubleshooting

For support operators, developers and infrastructure owners. Baseline: inspected source, 11 November 2026. Monitoring, backups, retention and support agreement: **Pending confirmation**. This document does not authorize production repairs or data changes.

Routine ownership

Agree a named owner and schedule for service billing/quota alerts, scheduler health, payment exceptions, stock reconciliation, enquiry response, contact/policy accuracy and dependency/security maintenance. The application contains operational signals; this does not establish an external alerting system or 24-hour support service.

Check worker heartbeat freshness/mode/key, unresolved payment/manual-review records and provider Dashboard health under approved access. Interpret a zero-work response as idle only when authentication, freshness and queue checks also succeed. Do not fabricate heartbeats or change LIVE flags to investigate unrelated UI problems.

Use safe request IDs and bounded diagnostic codes when escalating catalog/payment failures. Avoid environment dumps, service-account contents, authorization headers, reset links and full customer/provider payloads. Logs can contain business identifiers; redact before sharing.

Symptom guide

Symptom	First safe action	Escalation boundary
Products/categories missing	Review publication/current category visibility and safe API failure; retry once after connectivity recovery	Developer checks server setup/indexes; do not create fictional products
Upload/verification fails	Keep editor draft; retry verification where offered; confirm server media configuration	Cloudinary owner checks account/preset; do not expose signing secrets
Admin access denied	Confirm intended signed-in account and both claim/enabled record	Authorized owner reviews provisioning; no customer/profile privilege shortcut
Stale edit/conflict	Reload/review latest saved version before confirming	Do not overwrite unseen changes or create a second stock adjustment blindly
Payment pending after debit	Inspect existing order/status and trusted reconciliation path	Do not tell customer to pay again merely because navigation failed
Online checkout unavailable	Check safe mode/configuration eligibility and fresh matching heartbeat	Cloud/payment owner; COD still requires valid delivery and its own availability
Worker error/stale heartbeat	Check scheduler URL/method/protected header, timeout, deployed function and matching queue indexes	Developer; no manual heartbeat writes
Webhook accepted but status stale	Check event processing and provider reconciliation under correct mode/key	Acceptance 202 does not prove processing; do not manually mark captured
Coupon rejected	Check public/private code, dates, minimum, usage, exact Delivered-order eligibility and current source	Store owner reviews terms; do not bypass server validation
Points missing/held	Review order snapshot, Delivered eligibility, captured online evidence, ledger and return/refund review	No general balance editor; explicit repair needs separate approval
Delivery unavailable	Use Google setup troubleshooting	No production mock/flat/straight-line fallback
Tracking missing	Shipment details are optional; inspect actual saved order metadata	Manual carrier contact; no assumed carrier API
Enquiry has no email notification	Check Admin Enquiries storage/status	Storage/marked Replied is not proof of an outbound email service
GA4 empty	Consent/privacy signal, eligible environment/domain, measurement setting, blocked script and property settings	Never remove opt-in or install a duplicate tag as a shortcut

Payment and data incidents

Preserve the order, attempt, webhook and provider evidence. Establish whether the incident is display lag, unknown provider creation, failed attempt, captured payment, refund or mode/key mismatch. Record sanitized timestamps, deployment/commit, error code and customer-support action. Recovery uses the original operation and payment identity.

Credential rotation/mode switching can make older attempts require their original account configuration. Inventory, coupons and points are linked to reservation state; never release one independently just because a browser displayed failure. If provider refund allocation is unresolved, follow the existing review/freeze path. Do not use a COD manual record to manufacture an online refund.

Suspected secret exposure requires owner-led rotation, review of access/logs and controlled deployment/scheduler/webhook updates. Do not publish the old or replacement secret. Decide safe continuation of existing pending payments before changing active credentials.

Privileged scripts

The repository includes Admin provisioning, customer-directory rebuild, delivery-award repair and terminal TEST scheduling retirement tools. Their presence does not authorize execution. Before any run, separately review exact flags/guards, target project/environment, candidate IDs, default dry-run behaviour, affected fields, evidence preservation and rollback/acceptance plan with the owner. Do not run them during routine builds or to silence a failing test.

Backup and recovery

Actual automated backup/PITR/export configuration is **Pending confirmation**. A CSV export covers its selected business fields, not a complete system backup. Agree:

- Firestore configuration and business data coverage, retention and restore permissions.
- Firebase Authentication account recovery/export strategy, separate from Firestore profiles.
- Cloudinary asset recovery/ownership, Git repository and deployment/configuration recovery.
- Secure access to credential stores and scheduler/provider configuration; do not put secrets into the documentation backup.
- Recovery point/time objectives, isolated restore testing and incident approver.

Do not restore old financial databases over current provider activity or delete duplicate-looking evidence without a reconciliation plan. Application rollback and data restore are different procedures; see release guide.

Maintenance changes

Use a scoped branch/worktree or coordinated workflow; concurrent agents/editors sharing one checkout can overwrite unrelated work. Review diffs, select relevant existing checks once, and obtain release approval. Dependency upgrades and rule/index publication are separate reviewed changes. Keep current guides and verification evidence tied to the resulting commit.

Sources: `server/diagnostics.ts`, `server/payments/`, `server/loyalty.ts`, `server/loyaltyRepair.ts`, `src/loading/`, existing operational scripts. Reuse catalog diagnostics, payment QA, order recovery, coupon/loyalty audit. Their dated findings are historical, not newly verified incidents.